

Func, Action and Predicate Delegates in



C#.NET

Mr. Bangar Raju



Understanding Func, Action, and Predicate Delegates in C#

By Albert Harmon

Published April 28, 2017

🕒 4 min read

[Technology](#)



TLDR This blog post explores the three predefined generic delegates in C#: Func, Action, and Predicate. It explains their usage, provides code examples, and demonstrates how to simplify delegate usage with lambda expressions.

In this tutorial, we will delve into the world of delegates in C#, specifically focusing on three predefined generic delegates: Func, Action, and Predicate. These delegates are part of the Base Class Libraries and provide a streamlined way to work with methods that return values or perform actions.

What are Delegates?

Delegates in C# are type-safe function pointers that allow methods to be passed as parameters. They are particularly useful for defining callback methods and implementing event handling. In this post, we will explore how to use the predefined delegates to simplify our code.

Predefined Generic Delegates

The three predefined generic delegates we will cover are:

1. **Func**: Used for methods that return a value.
2. **Action**: Used for methods that do not return a value (void methods).
3. **Predicate**: Used for methods that return a Boolean value.

1. Func Delegate

The Func delegate is used when a method returns a value. It can take up to 16 input parameters and returns a single output. Here's how to

define and use a Func delegate:

Example: Using Func Delegate

```
public static double AddNums(int x, float y, double z) {  
    return x + y + z;  
}
```

```
Func<int, float, double, double> funcDelegate = AddNums;  
double result = funcDelegate.Invoke(1, 2.5f, 3.0);  
Console.WriteLine(result); // Output: 6.5
```

In this example, we define a method `AddNums` that takes three parameters and returns their sum. We then create a Func delegate that points to this method and invoke it with specific values.

2. Action Delegate

The Action delegate is used for methods that do not return a value. Similar to Func, it can also take up to 16 input parameters but does not provide an output.

Example: Using Action Delegate

```
public static void PrintSum(int x, float y, double z) {  
    Console.WriteLine(x + y + z);  
}
```

```
Action<int, float, double> actionDelegate = PrintSum;  
actionDelegate.Invoke(1, 2.5f, 3.0); // Output: 6.5
```

In this case, the `PrintSum` method performs a calculation and prints the result directly. We use an Action delegate to call this method.

3. Predicate Delegate

The Predicate delegate is specifically designed for methods that return a Boolean value. It takes a single input parameter and returns true or false.

Example: Using Predicate Delegate

```
public static bool CheckLength(string str) {  
    return str.Length > 5;  
}  
  
Predicate<string> predicateDelegate = CheckLength;  
bool status = predicateDelegate.Invoke("Hello");  
Console.WriteLine(status); // Output: False
```

Here, the `CheckLength` method checks if the length of a string is greater than five. We use a Predicate delegate to evaluate this condition.

Simplifying Delegate Usage with Lambda Expressions

In addition to using named methods, we can simplify our code by using lambda expressions. This allows us to define the logic inline without creating separate methods.

Example: Using Lambda Expressions with Delegates

```
Func<int, float, double, double> funcDelegate = (x, y, z) => x  
Console.WriteLine(funcDelegate.Invoke(1, 2.5f, 3.0)); // Output: 1
```

```
Action<int, float, double> actionDelegate = (x, y, z) => Console.WriteLine(x + y + z);  
actionDelegate.Invoke(1, 2.5f, 3.0); // Output: 6.5
```

```
Predicate<string> predicateDelegate = str => str.Length > 5;  
bool status = predicateDelegate.Invoke("Hello");  
Console.WriteLine(status); // Output: False
```



Using lambda expressions, we can define the behavior of our delegates in a more concise manner, making the code cleaner and easier to read.

Conclusion

In summary, the Func, Action, and Predicate delegates provide powerful tools for working with methods in C#. By leveraging these predefined delegates, we can simplify our code and improve its readability. Whether you are returning values, performing actions, or checking conditions, these delegates can help streamline your programming tasks.

Next time you need to define a delegate, consider using Func, Action, or Predicate to enhance your C# applications.